06/05/25

SQS (Simple Queue Service)

What it is:
A service to store and send messages between apps safely.

Use Case:
Online store puts orders in a queue, and workers handle them one by one.

Key Features:
- Stores messages
- Services work independently
- Handles many messages
- Two types: Standard & FIFO

Step-by-Step: Create an SQS Queue

Step 1: Go to SQS
- Sign in to AWS Console: https://console.aws.amazon.com
- Search for SQS in the search bar.
- Click Simple Queue Service.

Step 2: Create Queue
- Click "Create queue".
- Choose Standard (for most cases).
- Name your queue (e.g., MyTestQueue).
- Leave most settings as default (or adjust if needed).
- Click Create Queue at the bottom.

Step 3: Send a Message
- Select your new queue from the list.
- Click "Send and receive messages".
- Click "Send message".
- Type a sample message (e.g., Hello from SQS).
- Click Send message.

Step 4: Receive a Message
- Click "Poll for messages".

• You'll see the message you sent.
• You can also delete it after viewing.

🛒 Order Processing System with SQS – Step-by-Step Workflow

Step 1: Customer Places Order
• A user clicks "Buy Now" on a product.
• The frontend sends order details to the Order Service (backend).

Step 2: Order Service Sends Message to SQS
• The Order Service puts the order info (e.g., item, user ID, payment ID) into the SQS Queue.
📦 Now the message is safely stored in the queue.

Step 3: Worker Service Reads Message from SQS
• A Worker Service (could be a background app or AWS Lambda) keeps checking the queue.
• It receives the message from SQS.

Step 4: Process the Order
• The worker:
  ○ Verifies payment
  ○ Updates order status
  ○ Notifies shipping
  ○ Stores data in a database

Step 5: Delete Message from Queue
• After successful processing, the worker deletes the message from the queue.
• This prevents the same order from being processed again.

✅ Why Use SQS Here?
• Handles high traffic smoothly
• Keeps orders safe until processed
• Lets you retry if something fails
• Decouples order logic from payment, shipping, etc.

❌ Without SQS: What Can Go Wrong?

1. Direct Processing = Tight Coupling
- The Order Service directly calls Payment, Inventory, and Shipping services.
- If any one of them is slow or down, the whole process fails.

2. No Buffering
- If 1000 users place orders at once:
  - Without SQS: System may crash or slow down.
  - With SQS: Messages are queued and processed smoothly.

3. No Retry Logic
- If sending to Payment Service fails, the order is lost unless you manually handle retries.

4. Hard to Scale
- Without queues, scaling each service independently is difficult.
- One weak service can bring down the whole flow.

5. Poor Fault Tolerance
- Any temporary issue (e.g., network glitch, service outage) can cause order loss.

What is SNS (Simple Notification Service)?

SNS is a service that sends messages to many subscribers instantly.

It's like a megaphone: one message goes out to many listeners (SMS, Email, Lambda, HTTP, etc.).

Real-Time Use Case: Order Shipment Notification System

📦 Workflow: Notify Users When Order Is Shipped

🔁 Step-by-Step Workflow with SNS:

1. Order is Shipped
- The Shipping Service updates order status in the database.

2. Publish Message to SNS
- The Shipping Service publishes a message (e.g., "Order #123 Shipped") to an SNS Topic.

3. Subscribers Receive Notification
- The SNS Topic sends this message to:
  - ✅ User's Email (via email protocol)
  - ✅ User's Phone (via SMS)
  - ✅ Admin Dashboard (via HTTPS endpoint or Lambda)

○ ✅ Mobile App (via push notification)
All receivers get the same message instantly.

Why SNS is Useful Here:
• You don’t have to send messages manually to each user.
• Easy to add/remove subscribers.
• Reliable and scalable.

❌ What If SNS is Not Used?

Without SNS:
• The Shipping Service must manually send:
○ Email to user
○ SMS
○ Call other APIs
• If one fails (e.g., SMS provider is down), the entire flow is affected.
• More code, more bugs, more delay.

With SNS:
• Just publish once — SNS handles the rest reliably and in parallel.

🔄 Real-World Example with Load Balancer + SNS

Use Case: Server Health Monitoring

1. Load Balancer Monitors App Servers
• Application Load Balancer (ALB) checks if servers are healthy.

2. Server Goes Down
• ALB marks a target (server) as unhealthy.

3. ALB Triggers a CloudWatch Alarm
• CloudWatch detects the unhealthy server and triggers an alarm.

4. Alarm Publishes to SNS
• The CloudWatch alarm sends a message to SNS Topic.

5. SNS Sends Alerts
• SNS notifies:

○ Admin Email
○ Slack Webhook
○ Lambda to auto-restart server

✅ **Final Benefits of SNS:**
• Fast, parallel delivery
• Multiple formats (Email, SMS, Lambda, HTTP, SQS)
• Auto scaling & highly reliable
• Easy to plug into AWS monitoring (CloudWatch, ALB)

🔄 SNS vs SQS – Key Differences

Feature	SNS (Simple Notification Service)	SQS (Simple Queue Service)
Purpose	Broadcast messages to **many subscribers at once**	Queue messages to be **processed one by one**
Message Pattern	**Publish-Subscribe** (1-to-many)	**Point-to-Point** (1-to-1)
Delivery Targets	Email, SMS, Lambda, HTTP, SQS, etc.	Only to **consumers** that read from the queue
Delivery Type	**Push-based** – SNS pushes to subscribers	**Pull-based** – Services pull messages when ready
Message Storage	Does **not store messages** after delivery	Stores messages until they are processed & deleted
Use Case	Send notifications/alerts to multiple systems/users	Decouple services; buffer tasks to be processed reliably
Order Guarantee	No guarantee of message order	FIFO queue supports strict order (if needed)
Retry Logic	Basic retries, dead-letter topics	Built-in retries, visibility timeout, dead-letter queues

Fan-out Support	Yes – One message → Many subscribers	No – One message → One consumer